CN & SCALER · SESSION 3

One Process, Many Clients - Scaling Servers

select · epoll · CGI · FastCGI · and what your favourite server actually does

Week	Broad Topic
Week 1	Network programming 101
Week 2	Computer Networking Eagle Eye View
Week 3	Server Side Considerations for scale
Week 4	nginx deep dive
Week 5	Evolution of HTTP
Week 6	(Ab)using CDNs
Week 7	Economics of cloud tech
Week 8	Building for failures

One question - How do servers scale?

How does one box hold ten thousand clients at the same time?

01 **Repeat — select / epoll**
Handling multiple clients · timeouts, retries

02 **Intro to CGI / FastCGI**
How the server runs your code

03 **Isn't everything a web-server?**
Router admin · databases · IoT/MQTT · Kafka

04 **Limits & solutions**
ulimit & C10K · memory · context switches · queueing

05 **What does our favourite software do?**
Apache · HAProxy · Varnish · nginx

A thread per client is a bet you lose at ten thousand

Three ways to serve many clients — and only one of them survives contact with scale.

FORK PER REQUEST

A whole process copied for every client. Simple, isolated, and the most expensive thing you can do per connection.

THREAD PER REQUEST

Cheaper than fork — latency drops. It does not go away. Every thread still costs a stack before it does any work.

ONE LOOP, MANY SOCKETS

select or epoll. One thread watches every socket and only wakes for the ones that are ready. This is the one that scales.

WHAT ACTUALLY SHIPS

Most of the time we do neither purely — **we pre-fork or pre-thread a pool, and each worker runs its own select or epoll loop.** Isolation from the processes, concurrency from the loop.

fork(): a whole process per client

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(2026);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 1);

    while (1) {
        int client_fd = accept(server_fd, NULL, NULL);

        if (fork() == 0) {
            char buf[4096];
            int n;
            while ((n = read(client_fd, buf, sizeof(buf))) > 0) {
                write(client_fd, buf, n);
            }
            close(client_fd);
            return 0;
        }

        close(client_fd);
    }
}
```

WHAT IT COSTS

- A full process image per connection
- Page tables, fds, scheduler entry — all duplicated
- You pay it while the user waits

READ THE CODE

- listen(server_fd, 1) — a backlog of one
- The child echoes, then exits
- The parent closes its copy and loops

select(): one thread, every client

```c
int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(2026);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 16);

    int clients[FD_SETSIZE];
    int nclients = 0;

    while (1) {
        fd_set readfds;
        FD_ZERO(&readfds);
        FD_SET(server_fd, &readfds);
        int maxfd = server_fd;

        for (int i = 0; i < nclients; i++) {
            FD_SET(clients[i], &readfds);
            if (clients[i] > maxfd) maxfd = clients[i];
        }
```

```c
        select(maxfd + 1, &readfds, NULL, NULL, NULL);

        if (FD_ISSET(server_fd, &readfds)) {
            clients[nclients++] = accept(server_fd, NULL, NULL);
        }

        for (int i = 0; i < nclients; i++) {
            int fd = clients[i];
            if (!FD_ISSET(fd, &readfds)) continue;

            char buf[4096];
            int n = read(fd, buf, sizeof(buf));
            if (n <= 0) {
                close(fd);
                clients[i] = clients[--nclients];
                i--;
            } else {
                write(fd, buf, n);
            }
        }
    }
}
```

TWO LINES WORTH STOPPING ON

`clients[i] = clients[--nclients]; i--;` an O(1) delete. `select(..., NULL, NULL, NULL)` — that last NULL is the timeout.

select's ceiling is compiled in: FD_SETSIZE = 1024

That array in the previous slide is not a style choice — it is the hard limit.

```
int clients[FD_SETSIZE];
```

fd_set is a fixed-width bitmap. On glibc that width is 1024 bits — **descriptor 1024 cannot be represented at all.**

Same idea, different name: **kqueue** (BSD, macOS) · **IOCP** (Windows) · **io_uring** (modern Linux)

THE NUMBER TO REMEMBER

1024. It is FD_SETSIZE, it is your default ulimit, and it is the K in C10K. One wall, three names.

SELECT — O(n) EVERY CALL

- You rebuild the set on every iteration
- The kernel scans every fd, ready or not
- 10,000 idle sockets cost the same as 10,000 busy ones

EPOLL — O(ready)

- Register each fd once, with epoll_ctl
- epoll_wait returns only what actually fired
- Idle connections cost nothing to wait on

In production you don't pick one — you stack them

Pre-fork for isolation, pre-thread for cheapness, a loop inside each worker for scale.

AT STARTUP

Fork or spawn a pool of workers — often one per CPU core.

>

PER WORKER

Each runs its own select or epoll loop over many sockets.

>

ON ACCEPT

SO_REUSEPORT lets the kernel spread new connections across them.

>

RESULT

Crash isolation from processes, concurrency from the loop.

This is not a theoretical design. It is Apache’s event MPM, it is nginx’s master-and-workers, and it is what the Go runtime does for you invisibly — goroutines multiplexed onto OS threads by a scheduler sitting on top of epoll.

Timeouts are the client's half of the contract

The server may be busy reading, writing, or accepting. Never wait indefinitely.

In a select loop you are waiting on three different things:

READFDS
some new content to arrive

WRITEFDS
some socket allowing us to write

EXCEPTFDS
sockets being connected and closed

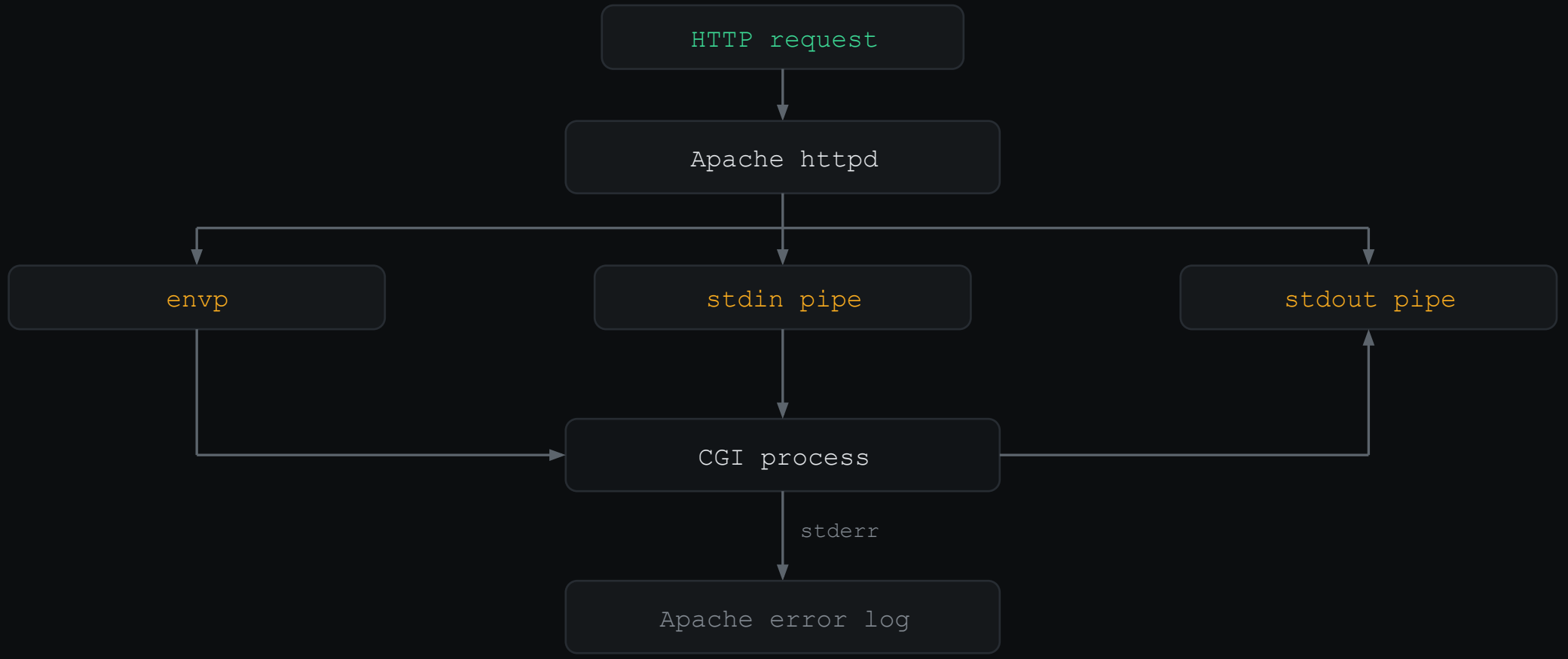
Each of those comes with a default timeout. **That is the last argument to select() — and in our code we passed NULL.**

```
select(maxfd + 1, &readfds, NULL, NULL, NULL);   /* readfds, writefds, exceptfds, timeout */
```

NULL timeout = block forever. One wedged client and the whole loop — every other connection on it — stops. On the client side the same rule: set a deadline, then retry with backoff.

CGI: every request forks a new copy of your program

Apache was the most popular HTTP server on earth, and this is how it ran your code.



Server side: wire up three pipes, then exec

Pseudo-code — and the real function in Apache that does it.

```
Set stdin  for the new process to a pipe/fd we control
Set stdout for the new process to another pipe/fd we control
Set stderr for the new process to yet another pipe/fd we control

Send text received from client   -> stdin
Send text written to stdout      -> client
Send text written to stderr      -> error log
```

THE REAL THING

`apache/httpd · modules/generators/mod_cgi.c L576 — run_cgi_child()`

Creates the child, attaches the three pipes, and sets a timeout on each so a hung script cannot hold the worker forever.

HTTP headers become environment variables

The translation layer between a protocol and a process.

```
Convert HTTP headers and verb to variables
E.g. REQUEST_METHOD = GET / POST,  QUERY_STRING = <things after ?>
Pass these as environment variables while creating the CGI process
```

THE CONVENTION

Header `User-Agent: curl` becomes `HTTP_USER_AGENT=curl`. Upper-case it, swap dashes for underscores, prefix with HTTP_. Every framework you have ever used still carries this shape.

THE REAL THING

`server/util_script.c  L409`
`ap_add_cgi_vars()`

Sets GATEWAY_INTERFACE, SERVER_PROTOCOL, REQUEST_METHOD, QUERY_STRING. ap_add_common_vars() above it does the rest.

The CGI program is just a program that prints

No server library. No framework. Read the environment, write to stdout.

```
CGI program

Get environment variable (REQUEST_METHOD)  // GET or POST
If POST - read from stdin
If GET  - parse another environment variable (QUERY_STRING)
Output "Content-Type: text/html" - two lines and text
```

TWO LINES

The header block, then a blank line, then the body. That blank line is the delimiter — the same framing question we ended on last session. Apache reads your headers up to it and streams everything after it to the client.

That is the whole contract. Any language that can read an environment variable and print to standard output can serve HTTP — which is exactly why the early web was written in Perl and shell.

Copy the whole program, then throw it away

We already said forking and executing is costly. CGI does it on every single hit.

THE COPY

The whole program image is created, run once, and discarded. That is what your shell does for every command you type — except your shell is not doing it a thousand times a second.

THE FIX PEOPLE TRIED

Compile the module INTO the server. mod_php, mod_perl. No new process per request — but now your application shares an address space with your web server, and one segfault takes both down.

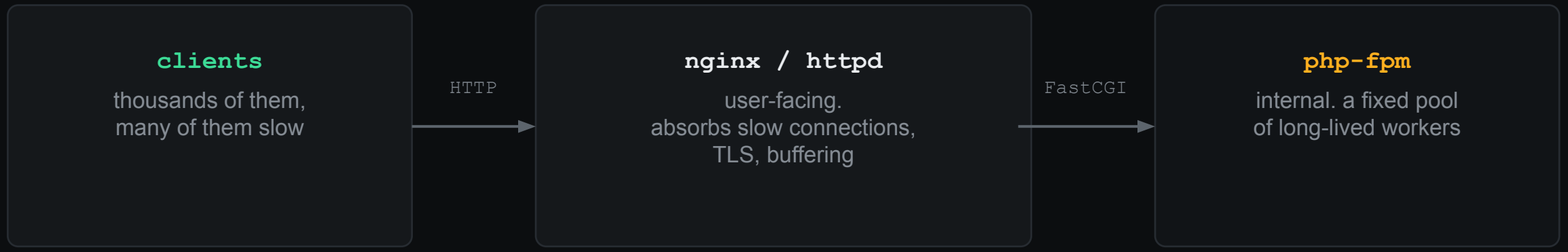
THE FIX THAT WON

Keep a long-lived process of your own and let the web server talk to it over a Unix or TCP socket, with a reduced vocabulary. That is FastCGI.

Notice the move: **take the setup cost off the request path and pay it once at startup.** Exactly what pre-forking did for connections in block 01.

FastCGI: keep the process, talk over a socket

You define an upstream. The user-facing server handles slow clients; an internal server does the work.



A MODIFIED HTTP, WITH LESS VOCABULARY

Same information — method, headers, query string, body — but binary, typed, and multiplexed. No parsing a text protocol twice.

WHAT YOU ACTUALLY WRITE

You may still get environment variables — but more often you get a **request object** with the same headers and query string, already decoded for you.

A FastCGI request is a stream of typed records

```
FCGI_BEGIN_REQUEST

FCGI_PARAMS
    REQUEST_METHOD = POST
    SCRIPT_FILENAME = /var/www/test.php
    QUERY_STRING = x=10
    CONTENT_TYPE = application/x-www-form-urlencoded
    CONTENT_LENGTH = 17
    HTTP_HOST = localhost
    HTTP_USER_AGENT = curl/...

FCGI_PARAMS
    <empty record means parameters finished>

FCGI_STDIN
    name=alice&age=20

FCGI_STDIN
    <empty record means stdin finished>
```

SAME NOUNS AS CGI

REQUEST_METHOD, QUERY_STRING, CONTENT_LENGTH — the CGI environment survived. Only the transport changed: records on a socket instead of envp on a fork.

STDIN IS THE BODY

name=alice&age=20 — seventeen bytes, which is exactly the CONTENT_LENGTH declared above it.

...and the response comes back the same way

```
FCGI_STDOUT
    Content-Type: text/html

    <html>...</html>

FCGI_STDOUT
    <empty>

FCGI_STDERR
    ...

FCGI_END_REQUEST
```

STDOUT GOES TO THE CLIENT

Headers, blank line, body — byte for byte what a CGI program printed. The web server parses off the headers and streams the rest.

STDERR GOES TO THE LOG

A separate record type, so your debug output can never corrupt the page. The same split as the pipes in the CGI diagram.

The empty record is the delimiter

Last session ended on framing: length or delimiter, no third option. Here they are, in one protocol.

LENGTH

Every record header carries its own content length. The reader knows exactly how many bytes to take before it takes them — so a body can contain any byte at all, including newlines and nulls.

`CONTENT_LENGTH = 17` does the same job one level up.

DELIMITER

A record of length zero means "that stream is finished". Not a byte pattern in the data — a structural marker that cannot collide with your content.

`The blank line after Content-Type` is the same trick in text.

SMS chose length. SMTP chose a dot. IMAP chose both. **FastCGI chose both too — and for the same reason.**

Servlets are the same idea wearing a JVM

Another way to avoid making a different program per request.

THE SERVER IS THE RUNTIME

Jetty and Tomcat act as the web server themselves. There is no separate httpd in front doing the forking — the JVM is already running your code.

PRE-THREADING

A pool of threads created at startup. A request is handed to an idle thread, which runs your servlet and returns to the pool. Exactly the pre-threading from block 01.

ISOLATION BY CONVENTION

Servlets are isolated by class loader and thread, not by process. Cheaper than fork — and one runaway servlet can still hurt its neighbours.

CGI, FastCGI, servlets, and every modern framework are answering one question: **how do we run user code per request without paying to create the world each time?**

Almost everything runs a web server

Not everything is one — and the difference decides which scaling tricks you get.

YES — IT SPEAKS HTTP

- Your Wi-Fi router admin page
- Modern agent runtimes and cloud tooling
- Most Node applications
- Firestore — the popular one
- CouchDB, which has had REST for years
- Kafka, if you put the REST proxy in front

NO — AND IT DOES NOT NEED TO

- Redis — its own line protocol, RESP
- MySQL — its own binary wire protocol
- MQTT brokers across the whole IoT estate
- Messenger backends — WhatsApp-shaped things
- Kafka natively — a binary protocol, not HTTP

The distinction earns a slide because **everything you are about to learn about scaling web servers assumes HTTP's properties.** Change the protocol and the advice changes with it.

HTTP is stateless. What you scale next may not be.

The protocol decides which scaling tricks are even available to you.

A WEB SERVER CAN HANG UP

Every request carries everything it needs. So any box can answer any request, you can close the connection when you like, and a load balancer can send the next one anywhere.

A DATABASE WANTS TO STAY

Redis or MySQL would rather hold a persistent connection than pay the cost of a TCP connect — plus a handshake, plus auth, plus session setup — on every query.

HORIZONTAL SCALING IS FREE-ISH

Add a server, add it to the pool, done. The statelessness is what makes that legal.

SO THE PROBLEM CHANGES SHAPE

Now the question is not requests per second. It is: how many persistent connections can one box hold at once? Which is the next block.

Four walls, and you will hit all of them

Every scaling technique in this course is a way of dodging one of these.

01 FILE DESCRIPTORS

A connection is a descriptor, and descriptors are rationed. Your default is still 1024.

02 MEMORY

A stack per thread, a buffer per connection. Paid before any work happens.

03 CONTEXT SWITCHES

Every read and write crosses the kernel boundary. So does every syscall.

04 THE QUEUE

Latency and throughput are not independent. Being slow costs you memory.

Let's talk about the kind of trouble web servers — and every other kind of server — get into.

C10K was 1999's problem. Your default ulimit is still 1024.

1999 · C10K

10,000

Dan Kegel names the problem: can one box hold ten thousand simultaneous persistent connections? At the time, no.

2013 · C10M

10,000,000

Robert Graham argues the kernel is the problem, not the solution. Bypass it and the number moves by three orders of magnitude.

TODAY

2–3M

People have demonstrated two to three million connections on a single server. The problem moved; the default did not.

```
$ ulimit -n
1024        # the soft limit on most distributions, unchanged since the problem was named
```

Every thread costs memory before it does any work

OS THREAD

8 MB default stack, reserved up front

10,000 threads reserves roughly 80 GB of address space before a single byte of your data exists.

GOROUTINE

2 KB initial stack, grown on demand

Four thousand times smaller. This single number is most of why Go took over this problem space.

AND UNDERNEATH, IT IS THE SAME LOOP

Goroutines are lightweight because the Go runtime multiplexes them onto a few OS threads using a netpoller built on epoll — and runs its own scheduler on top. You write blocking reads; the runtime parks you on a poll. **The more requests you handle at the same time, the more memory you need — the only question is how much per request.**

Every read and write is a trip across the kernel boundary

In fact, every system call is. At a million operations a second, the tax is the workload.

THE TAX

User space to kernel space and back: save state, switch page tables, validate, copy, return. Hundreds of nanoseconds each — and Spectre and Meltdown mitigations made it worse, not better.

BYPASS THE KERNEL

Some people have cleverly implemented TCP in user space and bypassed the kernel entirely for well-known internal networks. DPDK, netmap, mTCP, Seastar. You give up every tool you own and you get the packets.

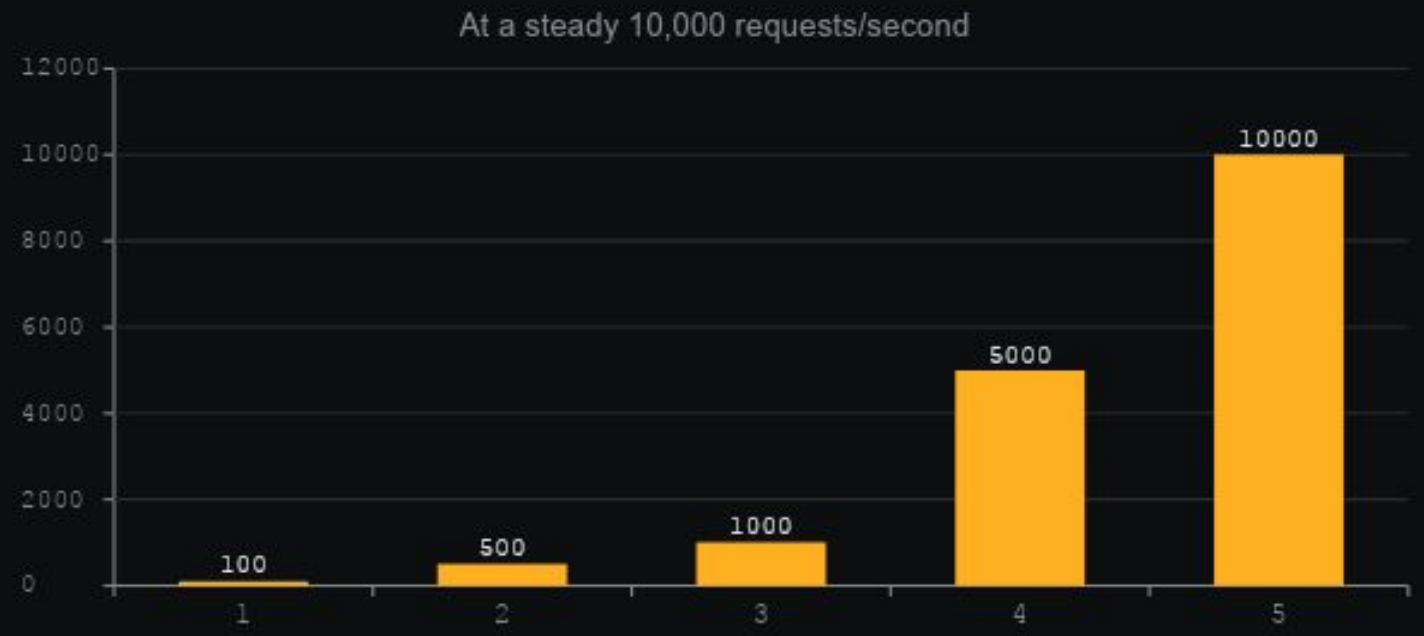
OR MAKE FEWER CALLS

sendfile() moves a file to a socket without a user-space round trip. io_uring submits a batch of operations through a shared ring instead of one syscall each.

Remember sendfile — it is the reason nginx serves static files the way it does, and we come back to it in the last block.

Concurrency isn't a setting — it's throughput × latency

```
connections in flight  =  requests per second  x  seconds per request
```



TEN TIMES THE COST

At 100 ms you hold a thousand connections. At one second, ten thousand — for identical traffic.

SLOWNESS IS A MEMORY BUG

Which is why high-latency work goes asynchronous: take it off the request path and the queue drains.

Apache: prefork → worker → event

It took twenty years, and it ended up at a loop too.

PREFORK MPM (Multi process module)

A pool of processes, one connection each. Bulletproof, compatible with every non-thread-safe module ever written, and the most expensive per connection.

>

WORKER MPM

Processes each running many threads. Cheaper per connection — but a thread is still pinned to a connection, including while it sits idle on keep-alive.

>

EVENT MPM

A dedicated listener thread per process parks idle keep-alive sockets on epoll and hands a worker thread only the sockets with actual work. The default on Unix today.

THE KEEP-ALIVE PROBLEM, IN ONE LINE

Under worker, a client that has finished its request but is holding the connection open still owns a thread. Event takes the socket back and gives the thread away. **That is our select loop, arriving inside Apache in 2012.**

HAProxy: a select loop from before epoll existed

THE TIMELINE EXPLAINS THE DESIGN

HAProxy arrives in 2001. epoll only lands in Linux 2.5.44 in late 2002. So it was largely select, because select was what there was. It moved to epoll later, and only became multi-threaded in 1.8, in 2017.

IT INTRODUCED THE LOAD BALANCER

For a lot of teams this was the first box that sat in front of the application servers and chose between them — and could switch between multiple upstream services.

ONE PROCESS, ENORMOUS THROUGHPUT

For most of its life a single event-driven process saturated a NIC. That is the loop argument made as loudly as it can be made.

AND IT ABSORBS CONNECTION CHURN

It can reduce the number of connects and disconnects on the upstream service by holding a pool open — the persistent-connection argument from block 03, applied by a proxy on your behalf.

Varnish: the first popular caching reverse proxy

SQUID CACHED FOR THE CLIENT

Squid was a caching proxy — it sat next to the users, on the way out. Varnish turned it around and put the cache in front of the origin, on the way in. Same mechanism, opposite direction, entirely different business.

STILL EVERYWHERE

Fastly's edge is built on Varnish, VCL and all. When you write a VCL snippet for a CDN in 2026, you are configuring a program written in 2006.

HOW IT ACTUALLY RUNS

An acceptor thread per listening point per pool. A worker thread per active connection, drawn from pools that default to a minimum of 100 and a maximum of 5000.

Idle descriptors — clients between requests, backends between fetches — are handed to a **waiter**, which is epoll on Linux and kqueue on BSD.

So: threads for the work, a loop for the waiting.
The same split Apache arrived at with the event MPM.

nginx: epoll plus sendfile — never copy the bytes at all

```
read(file_fd, buf, n);          /* disk -> kernel -> your buffer   */
write(sock_fd, buf, n);         /* your buffer -> kernel -> socket */

sendfile(sock_fd, file_fd, NULL, n);   /* kernel -> socket. you never touch it. */
```

MASTER AND WORKERS

One master, then one worker process per CPU core. Each worker is a single thread running one epoll loop over thousands of connections.

THE FIRST REVERSE PROXY

Then promptly abused for caching, load balancing, TLS termination, rate limiting and everything else. It became the default shape of an edge.

AND A THREAD POOL, LATER

Disk reads can block even in a loop. Since 1.7.11 nginx offloads those to a thread pool — because a page-fault on a cache miss stalls every connection on that worker.

All the options, side by side

Four teams, four starting points, thirty years apart.

	CONCURRENCY MODEL	BORN AS	THE IDEA IT POPULARISED	IN THE WILD
Apache	prefork → worker → event (listener thread + epoll)	1995 the web server	Modules, and CGI as a way to run anyone's code	Everywhere. Akamai started here
HAProxy	select, then epoll multi-threaded only since 1.8	2001 TCP/HTTP proxy	The load balancer as a tier of its own	The default LB in front of everything
Varnish	thread per active connection + epoll waiter for idle	2006 HTTP accelerator	Caching reverse proxy — Squid, turned around	Fastly's edge, VCL and all
nginx	master + worker per core one epoll loop each	2004 answer to C10K	sendfile: never copy the bytes into user space	The default edge. Broadpeak, and most CDNs

Read the second column top to bottom: **four independent teams, four starting points, and every one of them ended up with an event loop and a small pool of workers.**

Go and read the actual code

Every claim in the last block is a file you can open this evening.

apache/httpd	modules/generators/mod_cgi.c L576	run_cgi_child() — fork, three pipes, a timeout on each
apache/httpd	server/util_script.c L409	ap_add_cgi_vars() — HTTP headers become the environment
nginx/nginx	src/event/modules/ngx_epoll_module.c	The epoll loop itself
nginx/nginx	src/os/unix/ngx_linux_sendfile_chain.c	sendfile(), where the copy does not happen
haproxy/haproxy	src/ev_epoll.c	The poller, one file per platform — ev_select.c still sits beside it
varnishcache/varnish-cache	bin/varnishd/waiter/cache_waiter_epoll.c	The waiter that holds idle connections
golang/go	src/runtime/netpoll_epoll.go	Where your blocking read actually parks

---

Pick one. Reading a poller implementation end to end is a two-hour job and it changes how you read every server you meet afterwards — they are all the same shape once you have seen one.

Homework

Four things. Do at least the first two — they are the ones that leave a mark.

01

Break the select server.
Take the code from slide 05 and open 1,100 connections to it. Find where it dies and prove to yourself why. Then rewrite the same loop with epoll and do it again with 100,000.

02

Watch CGI cost you.
Write a CGI script in any language, run it under Apache with mod_cgi, and put ps or top on a second screen while you curl it in a loop. Count the processes appearing and disappearing.

03

Find the FastCGI records.
Run php-fpm behind nginx. tcpdump the loopback socket between them and find FCGI_BEGIN_REQUEST and the empty PARAMS record in the bytes.

04

Measure Little's Law on something you own.
Pick a service. Note its requests per second and its p50 latency, predict the connections in flight, then check with ss -s. Then make it slower on purpose and watch the prediction hold.

THE FOUR IDEAS

The loop is the architecture

01 AMORTISE THE SETUP

Pre-fork, pre-thread, FastCGI, connection pools, servlets. Every one is the same move: take the expensive setup off the request path and pay it once at startup.

02 NOTHING IS FREE PER CONNECTION

A descriptor, a stack, a buffer, a scheduler entry. Multiply by ten thousand before you choose a design — that multiplication is the whole discipline.

03 FRAMING IS STILL LENGTH OR DELIMITER

FastCGI uses a length inside a record and an empty record between them. Same two options as SMS and SMTP. There is never a third.

04 CONCURRENCY = THROUGHPUT × LATENCY

You cannot configure your way past this one. Getting faster is the only thing that reduces what you are holding.

Next session: we pick up at the load balancer and go looking for state.